

FOCUS_CHAIN_USER_GUIDE

Focus Chain System - Comprehensive User Guide

HelixCode Phase 2, Feature 3

Version: 1.0 Last Updated: November 7, 2025 Status: Complete

Table of Contents

1. [Introduction](#)
 2. [Quick Start](#)
 3. [Core Concepts](#)
 4. [API Reference](#)
 5. [Usage Examples](#)
 6. [Best Practices](#)
 7. [Integration Guide](#)
 8. [Troubleshooting](#)
 9. [FAQ](#)
-

Introduction

The Focus Chain System provides state management and context preservation across multiple interactions in HelixCode. It enables tracking of what users and LLMs are currently focused on, maintaining conversation context, and building intelligent focus-aware prompts.

What is a Focus?

A **Focus** represents a single point of attention in the development process - a file being edited, a bug being fixed, a feature being implemented, or any other work item that requires concentration.

What is a Focus Chain?

A **Focus Chain** is an ordered sequence of focuses that represents the flow of a conversation or work session. It maintains the history of what you've been working on and provides context for current work.

Key Features

- **Hierarchical Focus:** Parent-child relationships for complex tasks
 - **Multiple Focus Types:** Files, directories, tasks, errors, tests, functions, classes, packages
 - **Priority Levels:** Low, Normal, High, Critical
 - **Expiration Support:** Auto-cleanup of stale focuses
 - **Tags and Metadata:** Flexible categorization and custom data
 - **Thread-Safe:** Concurrent access support
 - **Chain Management:** Multiple parallel focus chains
 - **Context Preservation:** Shared and per-focus context
-

Quick Start

Creating a Simple Focus

```
package main

import (
    "dev.helix.code/internal/focus"
)

func main() {
    // Create a focus on a file
    myFocus := focus.NewFocus(focus.FocusTypeFile, "src/main.go")
    myFocus.Description = "Working on main entry point"
    myFocus.Priority = focus.PriorityHigh
    myFocus.AddTag("backend")

    // Add some context
    myFocus.SetContext("line", 42)
    myFocus.SetContext("function", "main")

    // Print focus info
    fmt.Println(myFocus.String())
}
```

Creating and Using a Chain

```
// Create a chain
chain := focus.NewChain("feature-implementation")

// Add focuses
chain.Push(focus.NewFocus(focus.FocusTypeTask, "implement-auth"))
chain.Push(focus.NewFocus(focus.FocusTypeFile, "auth/
    handler.go"))
chain.Push(focus.NewFocus(focus.FocusTypeTest, "auth/
    handler_test.go"))

// Navigate
current, _ := chain.Current()           // Current focus
prev, _ := chain.Previous()            // Go back
next, _ := chain.Next()                // Go forward

// Get specific focus
first, _ := chain.First()              // First focus
last, _ := chain.Last()               // Last focus

// Get info
fmt.Printf("Chain has %d focuses\n", chain.Size())
```

Using the Manager

```
// Create manager
manager := focus.NewManager()

// Create chains
authChain, _ := manager.CreateChain("auth-feature",
    true) // Set as active
testChain, _ := manager.CreateChain("tests", false)

// Push to active chain
manager.PushToActive(focus.NewFocus(focus.FocusTypeFile,
    "main.go"))

// Get current focus from active chain
current, _ := manager.GetCurrentFocus()

// Switch active chain
manager.SetActiveChain(testChain.ID)

// Get statistics
```

```
stats := manager.GetStatistics()
fmt.Printf("Total chains: %d\n", stats.TotalChains)
fmt.Printf("Total focuses: %d\n", stats.TotalFocuses)
```

Core Concepts

Focus Types

Type	Description	Example Use Case
FocusTypeFile	Single file	Editing main.go
FocusTypeDirectory	Directory	Working in src/api/
FocusTypeTask	Feature or task	Implementing authentication
FocusTypeError	Bug or error	Fixing bug #1234
FocusTypeTest	Test case	Writing unit tests
FocusTypeFunction	Specific function	Refactoring handleRequest()
FocusTypeClass	Class/struct	Modifying User struct
FocusTypePackage	Package/module	Updating auth package
FocusTypeProject	Entire project	Project overview
FocusTypeCustom	Custom type	Any custom focus

Priority Levels

```
const (
    PriorityLow      FocusPriority = 1 // Minor tasks
    PriorityNormal   FocusPriority = 5 // Regular work
    PriorityHigh     FocusPriority = 10 // Important tasks
    PriorityCritical FocusPriority = 20 // Critical issues
)
```

Hierarchical Focus

Focuses can have parent-child relationships:

```
// Create project focus
project := focus.NewFocus(focus.FocusTypeProject, "myapp")

// Create directory focus
srcDir := focus.NewFocus(focus.FocusTypeDirectory, "src")
project.AddChild(srcDir)
```

```
// Create file focus
mainFile := focus.NewFocus(focus.FocusTypeFile, "src/main.go")
srcDir.AddChild(mainFile)

// Navigate hierarchy
depth := mainFile.GetDepth()           // Returns 2
root := mainFile.GetRoot()             // Returns project
path := mainFile.GetPath()             // Returns [project,
    srcDir, mainFile]
```

Context and Metadata

Context: Dynamic, any-type data

```
focus.SetContext("line", 42)
focus.SetContext("cursor", CursorPosition{Line: 10, Col: 5})
focus.SetContext("edited", true)
```

```
value, ok := focus.GetContext("line") // Returns (42, true)
```

Metadata: String key-value pairs

```
focus.SetMetadata("author", "john")
focus.SetMetadata("created", "2025-11-07")
focus.SetMetadata("ticket", "PROJ-123")
```

```
author, ok := focus.GetMetadata("author") // Returns ("john",
    true)
```

Expiration

Focuses can have expiration times for automatic cleanup:

```
// Set expiration 1 hour from now
focus.SetExpiration(1 * time.Hour)
```

```
// Check if expired
if focus.IsExpired() {
    // Handle expired focus
}
```

```
// Chains auto-remove expired focuses
chain.Push(someExpiredFocus) // Automatically cleaned
chain.CleanExpired()         // Manual cleanup
```

API Reference

Focus API

Creation:

```
func NewFocus(focusType FocusType, target string) *Focus
func NewFocusWithPriority(focusType FocusType, target string,
    priority FocusPriority) *Focus
```

Validation:

```
func (f *Focus) Validate() error
```

Tags:

```
func (f *Focus) AddTag(tag string)
func (f *Focus) HasTag(tag string) bool
```

Context:

```
func (f *Focus) SetContext(key string, value interface{})
func (f *Focus) GetContext(key string) (interface{}, bool)
```

Metadata:

```
func (f *Focus) SetMetadata(key, value string)
func (f *Focus) GetMetadata(key string) (string, bool)
```

Expiration:

```
func (f *Focus) SetExpiration(duration time.Duration)
func (f *Focus) IsExpired() bool
```

Hierarchy:

```
func (f *Focus) AddChild(child *Focus)
func (f *Focus) RemoveChild(childID string) bool
func (f *Focus) GetDepth() int
func (f *Focus) GetRoot() *Focus
func (f *Focus) GetPath() []*Focus
func (f *Focus) FindChild(id string) *Focus
func (f *Focus) CountDescendants() int
```

Utility:

```
func (f *Focus) Touch()           // Update timestamp
func (f *Focus) Clone() *Focus    // Deep copy
func (f *Focus) String() string  // String representation
```

Chain API

Creation:

```
func NewChain(name string) *Chain
func NewChainWithSize(name string, maxSize int) *Chain
```

Stack Operations:

```
func (c *Chain) Push(focus *Focus) error
func (c *Chain) Pop() (*Focus, error)
func (c *Chain) Current() (*Focus, error)
```

Navigation:

```
func (c *Chain) Next() (*Focus, error)
func (c *Chain) Previous() (*Focus, error)
func (c *Chain) First() (*Focus, error)
func (c *Chain) Last() (*Focus, error)
func (c *Chain) Get(index int) (*Focus, error)
func (c *Chain) GetByID(id string) (*Focus, error)
func (c *Chain) SetCurrent(index int) error
```

Filtering:

```
func (c *Chain) GetRecent(n int) []*Focus
func (c *Chain) GetByType(focusType FocusType) []*Focus
func (c *Chain) GetByTag(tag string) []*Focus
func (c *Chain) GetByPriority(minPriority FocusPriority) []*Focus
```

Modification:

```
func (c *Chain) Remove(id string) error
func (c *Chain) Clear()
func (c *Chain) CleanExpired() int
```

Operations:

```
func (c *Chain) Merge(other *Chain) error
func (c *Chain) Split(index int) (*Chain, error)
func (c *Chain) Reverse()
func (c *Chain) Clone() *Chain
```

Info:

```
func (c *Chain) Size() int
func (c *Chain) IsEmpty() bool
func (c *Chain) String() string
```

Context:

```
func (c *Chain) SetContext(key string, value interface{})
func (c *Chain) GetContext(key string) (interface{}, bool)
func (c *Chain) SetMetadata(key, value string)
func (c *Chain) GetMetadata(key string) (string, bool)
```

Manager API

Creation:

```
func NewManager() *Manager
func NewManagerWithLimit(maxChains int) *Manager
```

Chain Management:

```
func (m *Manager) CreateChain(name string, setActive bool)
(*Chain, error)
func (m *Manager) CreateChainWithSize(name string, maxSize int,
setActive bool) (*Chain, error)
func (m *Manager) GetChain(id string) (*Chain, error)
func (m *Manager) GetActiveChain() (*Chain, error)
func (m *Manager) SetActiveChain(id string) error
func (m *Manager) DeleteChain(id string) error
func (m *Manager) GetAllChains() []*Chain
```

Focus Operations:

```
func (m *Manager) PushToActive(focus *Focus) error
func (m *Manager) GetCurrentFocus() (*Focus, error)
```

Queries:

```
func (m *Manager) FindChainsByName(nameSubstring string) []*Chain
func (m *Manager) GetRecentChains(n int) []*Chain
func (m *Manager) Count() int
func (m *Manager) GetStatistics() *ManagerStatistics
```

Maintenance:


```
func (m *Manager) CleanExpiredFocuses() int
func (m *Manager) MergeChains(targetID, sourceID string) error
func (m *Manager) Clear()
```

Import/Export:

```
func (m *Manager) ExportChain(id string) (*ChainSnapshot, error)
func (m *Manager) ImportChain(snapshot *ChainSnapshot, setActive
    bool) error
```

Callbacks:

```
func (m *Manager) OnCreate(callback ChainCallback)
func (m *Manager) OnDelete(callback ChainCallback)
func (m *Manager) OnActivate(callback ChainCallback)
```

Usage Examples

Example 1: Development Workflow

```
// Create manager and chain for a feature
manager := focus.NewManager()
chain, _ := manager.CreateChain("user-auth-feature", true)

// 1. Start with high-level task
task := focus.NewFocusWithPriority(
    focus.FocusTypeTask,
    "Implement user authentication",
    focus.PriorityHigh,
)
task.AddTag("feature")
task.AddTag("authentication")
manager.PushToActive(task)

// 2. Focus on specific file
file := focus.NewFocus(focus.FocusTypeFile, "auth/handler.go")
file.SetContext("line", 50)
file.SetContext("function", "Login")
manager.PushToActive(file)

// 3. Encounter an error
bug := focus.NewFocusWithPriority(
    focus.FocusTypeError,
    "Login returns 500 error",
```

```

        focus.PriorityCritical,
    )
    bug.SetMetadata("error_code", "500")
    bug.SetMetadata("stack_trace", "...")
    manager.PushToActive(bug)

    // 4. After fixing, move to tests
    test := focus.NewFocus(focus.FocusTypeTest, "auth/
        handler_test.go")
    test.AddTag("unit-test")
    manager.PushToActive(test)

    // Review workflow
    current, _ := manager.GetCurrentFocus()
    fmt.Printf("Currently: %s\n", current.Target)

    // Go back through history
    chain.Previous()
    chain.Previous()
    prev, _ := chain.Current()
    fmt.Printf("Was working on: %s\n", prev.Target)

```

Example 2: Code Review Session

```

manager := focus.NewManager()
reviewChain, _ := manager.CreateChain("code-review-pr-123", true)

// Get list of changed files from PR
changedFiles := []string{
    "src/api/handler.go",
    "src/api/middleware.go",
    "src/api/validator.go",
}

// Create focus for each file
for _, file := range changedFiles {
    f := focus.NewFocus(focus.FocusTypeFile, file)
    f.AddTag("code-review")
    f.AddTag("pr-123")
    f.SetMetadata("reviewer", "john")
    reviewChain.Push(f)
}

// Review each file

```

```

for reviewChain.CurrentIdx >= 0 {
    current, _ := reviewChain.Current()

    // Simulate review
    fmt.Printf("Reviewing: %s\n", current.Target)

    // Add review notes
    current.SetContext("status", "approved")
    current.SetContext("comments", []string{
        "LGTM",
        "Good error handling",
    })

    // Move to next
    reviewChain.Next()
}

// Get all reviewed files
allFocuses := reviewChain.GetByTag("code-review")
fmt.Printf("Reviewed %d files\n", len(allFocuses))

```

Example 3: Bug Fixing Session

```

manager := focus.NewManager()
bugChain, _ := manager.CreateChain("fix-bug-1234", true)

// Start with bug report
bug := focus.NewFocusWithPriority(
    focus.FocusTypeError,
    "Users can't login after password reset",
    focus.PriorityCritical,
)
bug.SetMetadata("ticket", "BUG-1234")
bug.SetMetadata("reporter", "customer-support")
bug.SetMetadata("impact", "high")
bugChain.Push(bug)

// Investigate related files
suspects := []string{
    "auth/password.go",
    "auth/session.go",
    "db/user_repo.go",
}

```

```

for _, file := range suspects {
    f := focus.NewFocus(focus.FocusTypeFile, file)
    f.AddTag("investigation")
    bugChain.Push(f)
}

// Found the issue in one file
problemFile, _ := bugChain.GetByID("specific-focus-id")
problemFile.SetContext("issue_found", true)
problemFile.SetContext("root_cause",
    "session not refreshed after password reset")

// Add fix focus
fix := focus.NewFocus(focus.FocusTypeFile, "auth/session.go")
fix.SetContext("fix_applied", true)
fix.SetMetadata("commit", "abc123")
bugChain.Push(fix)

// Add test to verify fix
test := focus.NewFocus(focus.FocusTypeTest, "auth/
    session_test.go")
test.AddTag("bug-fix")
test.AddTag("regression-test")
bugChain.Push(test)

// Summary
fmt.Printf("Bug fix workflow: %d steps\n", bugChain.Size())
investigationFocuses := bugChain.GetByTag("investigation")
fmt.Printf("Investigated %d files\n", len(investigationFocuses))

```

Example 4: Multi-Chain Management

```

manager := focus.NewManager()

// Active development chain
devChain, _ := manager.CreateChain("active-development", true)
devChain.Push(focus.NewFocus(focus.FocusTypeFile, "main.go"))

// Background research chain
researchChain, _ := manager.CreateChain("research", false)
researchChain.Push(focus.NewFocus(focus.FocusTypeTask,
    "investigate-caching"))

// Code review chain
reviewChain, _ := manager.CreateChain("code-reviews", false)

```

```

reviewChain.Push(focus.NewFocus(focus.FocusTypeTask, "review-
pr-45"))

// Switch between chains as needed
manager.SetActiveChain(reviewChain.ID)
// Do review work...

manager.SetActiveChain(devChain.ID)
// Resume development...

// Get statistics
stats := manager.GetStatistics()
fmt.Printf("Managing %d chains\n", stats.TotalChains)
fmt.Printf("Total focus points: %d\n", stats.TotalFocuses)
fmt.Printf("Average focuses per chain: %.1f\n",
    stats.AverageFocusesPerChain)

// Find chains by name
authChains := manager.FindChainsByName("auth")
fmt.Printf("Found %d auth-related chains\n", len(authChains))

```

Best Practices

1. Use Appropriate Focus Types

Choose the right focus type for your work:

```

// x Good: Specific types
task := focus.NewFocus(focus.FocusTypeTask, "implement-login")
file := focus.NewFocus(focus.FocusTypeFile, "auth.go")
bug := focus.NewFocus(focus.FocusTypeError, "crash-on-startup")

// Bad: Everything as custom
everything := focus.NewFocus(focus.FocusTypeCustom, "work")

```

2. Set Meaningful Priorities

Use priority levels consistently:

```

// Critical: Production issues, security vulnerabilities
critical := focus.NewFocusWithPriority(
    focus.FocusTypeError,
    "SQL injection in login",
    focus.PriorityCritical,

```

```

)

// High: Important features, blocking bugs
high := focus.NewFocusWithPriority(
    focus.FocusTypeTask,
    "User registration",
    focus.PriorityHigh,
)

// Normal: Regular development work
normal := focus.NewFocus(focus.FocusTypeFile, "utils.go")

// Low: Nice-to-have improvements
low := focus.NewFocusWithPriority(
    focus.FocusTypeTask,
    "Update README",
    focus.PriorityLow,
)

```

3. Use Tags for Organization

Tags enable powerful filtering:

```

focus := focus.NewFocus(focus.FocusTypeFile, "api.go")
focus.AddTag("backend")
focus.AddTag("api")
focus.AddTag("refactoring")
focus.AddTag("in-progress")

```

```

// Later: Find all backend refactoring work
backendWork := chain.GetByTag("backend")
inProgress := chain.GetByTag("in-progress")

```

4. Leverage Context for State

Store dynamic data in context:

```

focus := focus.NewFocus(focus.FocusTypeFile, "handler.go")

// Editor state
focus.SetContext("line", 42)
focus.SetContext("column", 10)
focus.SetContext("selected_text", "function handleRequest")

// Work state

```

```
focus.SetContext("last_edit", time.Now())
focus.SetContext("saved", true)
focus.SetContext("test_status", "passing")
```

5. Use Metadata for Persistent Info

Metadata for searchable string data:

```
focus.SetMetadata("author", "john")
focus.SetMetadata("ticket", "PROJ-123")
focus.SetMetadata("branch", "feature/auth")
focus.SetMetadata("reviewed_by", "jane")
```

6. Set Expiration for Temporary Focus

Use expiration for time-sensitive focuses:

```
// Short-lived focus for current editing session
editFocus := focus.NewFocus(focus.FocusTypeFile, "temp.go")
editFocus.SetExpiration(30 * time.Minute)

// Bug fix has 1-week deadline
bugFocus := focus.NewFocus(focus.FocusTypeError, "memory-leak")
bugFocus.SetExpiration(7 * 24 * time.Hour)
```

7. Organize with Hierarchy

Use parent-child for complex work:

```
// Feature focus
feature := focus.NewFocus(focus.FocusTypeTask, "user-
    authentication")

// Sub-components
login := focus.NewFocus(focus.FocusTypeFile, "login.go")
session := focus.NewFocus(focus.FocusTypeFile, "session.go")
tests := focus.NewFocus(focus.FocusTypeTest, "auth_test.go")

feature.AddChild(login)
feature.AddChild(session)
feature.AddChild(tests)

// Navigate hierarchy
allSubtasks := feature.Children
totalWork := feature.CountDescendants()
```

8. Regular Cleanup

Remove expired and completed focuses:

```
// Periodic cleanup
cleanedCount := chain.CleanExpired()
fmt.Printf("Removed %d expired focuses\n", cleanedCount)

// Or let manager clean all chains
totalCleaned := manager.CleanExpiredFocuses()
```

9. Use Callbacks for Integration

Register callbacks for automation:

```
manager.OnCreate(func(chain *Chain) {
    log.Printf("New chain created: %s\n", chain.Name)
    // Persist to database, send notification, etc.
})

manager.OnActivate(func(chain *Chain) {
    log.Printf("Switched to chain: %s\n", chain.Name)
    // Update UI, change context, etc.
})

manager.OnDelete(func(chain *Chain) {
    log.Printf("Chain deleted: %s\n", chain.Name)
    // Cleanup resources, archive, etc.
})
```

10. Export/Import for Persistence

Save and restore chains:

```
// Export chain
snapshot, _ := manager.ExportChain(chain.ID)

// Save to file/database
data, _ := json.Marshal(snapshot)
os.WriteFile("chain.json", data, 0644)

// Later: Load from file
data, _ := os.ReadFile("chain.json")
var snapshot *focus.ChainSnapshot
json.Unmarshal(data, &snapshot)
```



```
// Import
manager.ImportChain(snapshot, false)
```

Integration Guide

Integration with Task System

```
// When creating a task, create corresponding focus
func createTaskWithFocus(manager *focus.Manager, taskName
    string) {
    // Create task (your existing code)
    task := createTask(taskName)

    // Create focus
    taskFocus := focus.NewFocusWithPriority(
        focus.FocusTypeTask,
        taskName,
        focus.PriorityHigh,
    )
    taskFocus.SetMetadata("task_id", task.ID)

    // Push to active chain
    manager.PushToActive(taskFocus)
}
```

Integration with LLM Context

```
// Build LLM prompt with focus context
func buildPromptWithFocus(manager *focus.Manager) string {
    chain, _ := manager.GetActiveChain()
    recent := chain.GetRecent(5)

    prompt := "You are currently working on:\n\n"

    for i, f := range recent {
        prompt += fmt.Sprintf("%d. %s (%s)\n", i+1, f.Target,
            f.Type)
        if f.Description != "" {
            prompt += fmt.Sprintf("    %s\n", f.Description)
        }
    }
}
```

```

current, _ := chain.Current()
prompt += fmt.Sprintf("\nCurrent focus: %s\n",
    current.Target)

return prompt
}

```

Integration with File Editor

```

// Update focus when file changes
func onFileChange(manager *focus.Manager, filePath string, line
    int) {
    // Get or create focus for file
    currentFocus, err := manager.GetCurrentFocus()

    if err != nil || currentFocus.Target != filePath {
        // Create new focus
        fileFocus := focus.NewFocus(focus.FocusTypeFile,
            filePath)
        manager.PushToActive(fileFocus)
        currentFocus = fileFocus
    }

    // Update context
    currentFocus.SetContext("line", line)
    currentFocus.SetContext("last_edit", time.Now())
    currentFocus.Touch()
}

```

Troubleshooting

Issue: Focus Not Found in Chain

Problem: GetByID() returns error

Solution:

```

// Check if focus exists
focus, err := chain.GetByID(id)
if err != nil {
    // Focus might have been removed or expired
    allFocuses := chain.GetAll()
    fmt.Printf("Available focuses: %d\n", len(allFocuses))
}

```

Issue: Chain at Maximum Size

Problem: Push fails when chain is full

Solution:

```
// Create chain with larger size
chain := focus.NewChainWithSize("work", 1000)

// Or use unlimited size (0)
chain.MaxSize = 0
```

Issue: Expired Focuses Not Removed

Problem: Expired focuses still in chain

Solution:

```
// Explicitly clean
removed := chain.CleanExpired()
fmt.Printf("Removed %d expired focuses\n", removed)

// Or use manager to clean all
manager.CleanExpiredFocuses()
```

FAQ

Q: When should I use Focus vs Context vs Metadata?

A: - **Focus:** Represents what you're working on (file, task, bug) - **Context:** Dynamic runtime data (line number, cursor position, state) - **Metadata:** Persistent searchable strings (author, ticket ID, branch name)

Q: Should I create a new chain for each session?

A: It depends on your workflow. Common patterns: - One chain per feature/task - One chain per work session (day/week) - Multiple chains for parallel work streams

Q: How many focuses should a chain have?

A: Typical chains have 5-20 focuses. Use MaxSize if you need to limit. Remove completed focuses periodically.

Q: Can I share chains between users?

A: Yes, use Export/Import:

```
snapshot, _ := manager1.ExportChain(chainID)
manager2.ImportChain(snapshot, false)
```

Q: How do I prevent memory leaks?

A: Use expiration and regular cleanup:

```
// Set expiration on focuses
focus.SetExpiration(1 * time.Hour)
```

```
// Regular cleanup
chain.CleanExpired()
manager.CleanExpiredFocuses()
```

```
// Delete unused chains
manager.DeleteChain(oldChainID)
```

Q: Can I use Focus Chain in concurrent code?

A: Yes! Manager operations are thread-safe:

```
manager := focus.NewManager()
```

```
go func() {
    manager.CreateChain("worker-1", false)
}()
```

```
go func() {
    manager.CreateChain("worker-2", false)
}()
```

Conclusion

The Focus Chain System provides a powerful way to manage context and state across development sessions. By leveraging focuses, chains, and the manager, you can build intelligent, context-aware development tools that remember what users are working on and provide relevant assistance.

Key Takeaways

1. **Use appropriate focus types** for different work items
2. **Set priorities** to distinguish importance
3. **Leverage tags** for organization and filtering
4. **Use context for runtime state**, metadata for persistent info
5. **Set expiration** for temporary focuses

6. **Regular cleanup** prevents memory leaks
 7. **Callbacks enable integration** with existing systems
 8. **Thread-safe** operations support concurrent access
-

Document Version: 1.0 **Last Updated:** November 7, 2025 **Next Feature:** Hooks
System (Phase 2, Feature 4)